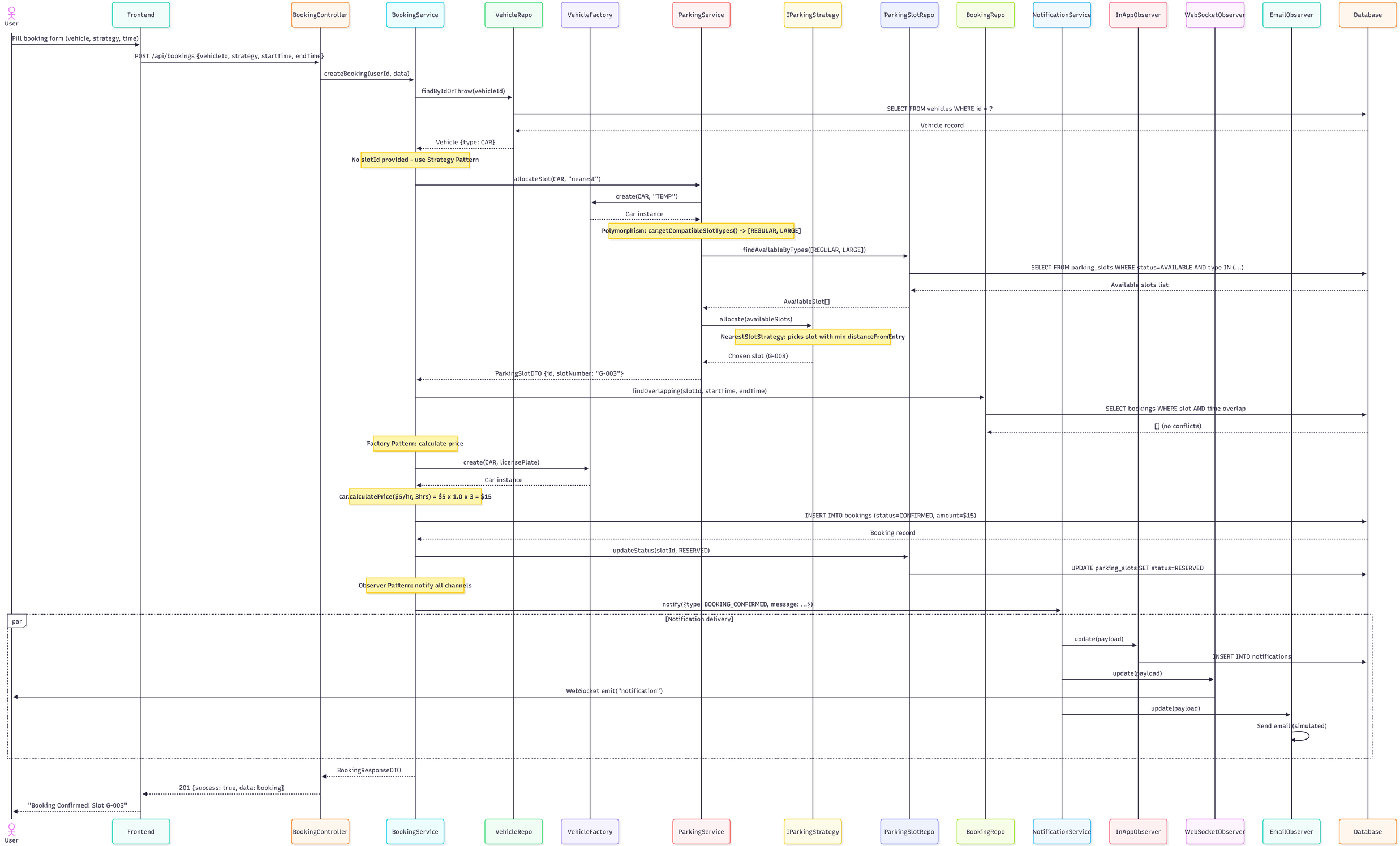
User
Frontend
BookingController
BookingService
VehicleRepo
VehicleFactory
ParkingService
IParkingStrategy
ParkingSlotRepo
BookingRepo
NotificationService
InAppObserver
WebSocketObserver
EmailObserver
Database
Fill booking form (vehicle, strategy, time)
POST /api/bookings {vehicleId, strategy, startTime, endTime}
createBooking(userId, data)
findByIdOrThrow(vehicleId)
SELECT FROM vehicles WHERE id = ?
Vehicle record
Vehicle {type: CAR}
No slotId provided - use Strategy Pattern
allocateSlot(CAR, "nearest")
create(CAR, "TEMP")
Car instance
Polymorphism: car.getCompatibleSlotTypes() -> [REGULAR, LARGE]
findAvailableByTypes([REGULAR, LARGE])
SELECT FROM parking_slots WHERE status=AVAILABLE AND type IN (...)
Available slots list
AvailableSlot[]
allocate(availableSlots)
NearestSlotStrategy: picks slot with min distanceFromEntry
Chosen slot (G-003)
ParkingSlotDTO {id, slotNumber: "G-003"}
findOverlapping(slotId, startTime, endTime)
SELECT bookings WHERE slot AND time overlap
[] (no conflicts)
Factory Pattern: calculate price
create(CAR, licensePlate)
Car instance
car.calculatePrice($5/hr, 3hrs) = $5 x 1.0 x 3 = $15
INSERT INTO bookings (status=CONFIRMED, amount=$15)
Booking record
updateStatus(slotId, RESERVED)
UPDATE parking_slots SET status=RESERVED
Observer Pattern: notify all channels
notify({type: BOOKING_CONFIRMED, message: ...})
[Notification delivery]
par
update(payload)
INSERT INTO notifications
update(payload)
WebSocket emit("notification")
update(payload)
Send email (simulated)
BookingResponseDTO
201 {success: true, data: booking}
"Booking Confirmed! Slot G-003"